Practical Session 3 : NLP Pipeline - Part 1

Tokenisation → Stopwords → Normalisation → Vectorisation → L2 Normalisation

Dr. BENDIABDALLAH M.H
Université Belhadj BOUCHAIB - Ain Temouchent

Module: Natural Language Processing (NLP)

Learning Outcomes

By the end of this practical session, you will be able to:

- Load a long text from a **separate .txt file**.
- Apply basic and rule-based tokenisation.
- Remove stopwords and justify the choice (including negations).
- Normalise words using stemming or lemmatisation.
- Vectorise text using `CountVectorizer` (unigrams and (1,2)-grams).
- Apply **L2 normalisation** to compare text segments using cosine similarity.

Environment Setup (Installations & Downloads)

1. Python Environment

Make sure you are using:

- Python 3.9 or higher
- Jupyter Notebook or JupyterLab

2. Required Python Libraries

Install the required libraries using **pip**:

```
conda install numpy pandas scikit-learn matplotlib
pip install nltk
```

3. NLTK Data Downloads (Run Once in Notebook)

The following linguistic resources are required for stopwords and lemmatisation.

```
import nltk

nltk.download("stopwords")
nltk.download("wordnet")
nltk.download("omw-1.4")
nltk.download("averaged_perceptron_tagger")
```

Dr. BENDIABDALLAH M.H

4. Verify Installation

Run the following cell to ensure everything works correctly:

```
import numpy
import pandas
import sklearn
import nltk
import matplotlib

print("All libraries successfully installed.")
```

---

1 Step 0 — Load the Long Text from a Separate File (.txt)

0A. Prepare the file

Create a text file named for example: `ai_text.txt` and paste the AI text inside it.

0B. Read the text in Python

```
# File path (same folder as your notebook is easiest)
FILE_PATH = "ai_text.txt"

with open(FILE_PATH, "r", encoding="utf-8") as f:
    AI_TEXT = f.read()

print("Number of characters:", len(AI_TEXT))
print("Preview:\n", AI_TEXT[:400], "...")
```

0C. Split the large text into multiple documents

We treat each paragraph as a **document** (needed for a document-term matrix).

```
import re

docs = [p.strip() for p in AI_TEXT.strip().split("\n\n") if p.strip()]

print("Number of documents (paragraphs):", len(docs))
for i, d in enumerate(docs, 1):
    print(f"\n--- Document {i} preview ---\n{d[:250]}...")
```

2 Step 1 — Tokenisation (basic vs rule-based)

1A. Basic tokenisation (whitespace)

```
def whitespace_tokenise(text: str):
    return text.split()

print("Whitespace tokens (first 40):")
print(whitespace_tokenise(docs[0])[:40])
```

Dr. BENDIABDALLAH M. H

1B. Rule-based tokenisation (regex)

```
def rule_based_tokenise(text: str):
    return re.findall(r"\w+|[^\w\s]", text)

print("Rule-based tokens (first 60):")
print(rule_based_tokenise(docs[0])[:60])
```

3 Step 2 — Stopwords Removal (justify your choice)

We remove frequent function words, but we keep negations because they change meaning.

```
import nltk
from nltk.corpus import stopwords

# One-time download (run once if needed)
# nltk.download("stopwords")

stop_words = set(stopwords.words("english"))

# Recommended: keep negations
negations_to_keep = {"no", "not", "nor", "never"}
stop_words = stop_words - negations_to_keep

def remove_stopwords(tokens, stopset):
    return [t for t in tokens if t.lower() not in stopset]

tokens_demo = [t.lower() for t in rule_based_tokenise(docs[0]) if t.isalpha()]
tokens_no_sw = remove_stopwords(tokens_demo, stop_words)

print("Before stopwords:", tokens_demo[:30])
print("After stopwords :", tokens_no_sw[:30])
```

Question :

- Print the list of removed stopwords.

```
# ANSWER : ...
```

4 Step 3 — Normalisation (stemming or lemmatisation)

Choose ONE method for the rest of this TP.

Question (answer in code):

- Apply both normalisation methods (stemming and lemmatisation) to `tokens_no_sw` and compare the results.

```
# ANSWER : ...
```

Dr. BENDIABDALLAH M.H